Choosing a Local Language Model for Automated Security Alert Triage

Benchmark Report

Ayman Djioui
Badr Jakout
SOC Internship, Devoteam

August 2026

1 What this document is about

Our internship project builds a small Security Operations Center in a lab network. A firewall and an intrusion detection system watch the traffic, the logs go into Elasticsearch, and when something suspicious appears an alert is created automatically in a case management tool called TheHive.

The problem with this setup is the same problem real SOCs have. The system produces far more alerts than anyone can read. Most of them are harmless. Somebody still has to look at each one and decide whether it matters. That first pass is called Tier-1 triage and it is slow, repetitive work.

We wanted to know whether a language model running on our own machine could do that first pass. Not to replace the analyst, but to write a draft assessment so a human starts from something instead of from nothing.

The model has to run locally. Sending security alerts from a client network to an external API is not acceptable, because those alerts contain internal addresses, hostnames and attack details. Everything described here runs on hardware we control.

This document explains how we chose the model, what data we tested it on, and what we found. The short version is that the smallest model we tested won, and it kept winning even after we gave it much larger competitors and better hardware.

2 What we ask the model to do

For each alert the model must return four things:

- **Is this a false positive?** True or false.
- **Severity.** One of low, medium, high, critical.
- **MITRE ATT&CK technique.** The identifier of the attack technique, taken from a fixed list, or N/A when no technique applies.
- **Recommended action.** A short sentence for the analyst.

The answer must come back as structured data, not as a paragraph of text, because the rest of our pipeline reads it as JSON. We enforce this with a feature called schema-constrained decoding,

which restricts the model so that it can only produce output matching a shape we define in advance.

One point matters for everything that follows. The model never decides whether to block anything. It produces an assessment. A separate piece of ordinary code decides whether any action is allowed. We made that separation after an early test where a model recommended blocking one of our own internal Windows machines because it was sending normal network discovery traffic. That result is what convinced us the decision had to sit in code rather than in the model.

3 The evaluation data

To compare models we needed a fixed set of alerts with known correct answers. We built seven test cases from our own lab. They are real events, captured from the running system, not invented examples.

3.1 Why seven cases, and how they were chosen

The cases were chosen to cover the different log sources we actually collect and, more importantly, to include the specific mistakes we expected a model to make. Three of them are traps. Two look dangerous but are harmless. One looks harmless but is a genuine attack. A model that only ever answers "this is an attack" scores badly on this set, which is exactly what we wanted.

Table 1: The seven evaluation cases

Case	Source	What it is	Correct answer
01 testmyids	Suricata IDS	Signature fires on a harmless test request to testmyids.com	not FP, medium
02 ssdp	Suricata IDS	Internal Windows host sending normal SSDP discovery traffic	**false positive**, low
03 attack resp.	Suricata IDS	External host, response pattern to an internal LAN machine	not FP, medium
04 powershell	Suricata IDS	Internal host with a PowerShell user agent going outbound	**false positive**, low
05 sqli	Juice Shop web logs	Login request with the payload `' OR 1=1-`	not FP, critical, T1190
06 cred dump	Windows Sysmon	`reg save HKLM\SAM` run by Administrator	not FP, high, T1003.002
07 nmap scan	Suricata IDS	Port scan against the exposed web application in the DMZ	not FP, medium, T1046

Cases 02 and 04 are the important ones. Both involve internal machines doing normal things that happen to match an intrusion detection signature. In a real SOC these are the alerts that waste the most time, and they are also the ones where a wrong automated decision does the most damage, because blocking an internal host takes part of the network offline.

Case 05 is the opposite trap. A single quote in a login form is easy to miss, and it is a real authentication bypass attempt against our deliberately vulnerable web application.

3.2 How we score

Each case is worth three points, one for each judgement we can check objectively:

- false positive, correct or not

- severity, correct or not
- MITRE technique, correct or not

Seven cases times three points gives a maximum of **21**. We also record whether the returned JSON was valid, and how long the answer took.

The recommended action is not scored. There is no single correct wording for it, so scoring it would mean grading a sentence by opinion, and that would make the whole benchmark subjective.

3.3 Honest limits of this data

Seven cases is a small set. One case is worth almost five percent of the total score, so small differences between models are not meaningful. We treat this benchmark as good enough to rank models clearly apart, and not good enough to argue about one or two points.

The test set was also written by the same person who later tuned the prompt. That is a real source of bias and we state it rather than hide it.

4 Experiment 1: comparing five models

The first question was simply which model to use. We tested five, chosen to cover a range of sizes and different developers. At this stage the prompt was plain, with no examples given to the model.

We ran this experiment twice on different hardware, first on a virtual machine on a laptop using only the processor, and later on a cloud machine with a graphics card. The models and the test cases were identical in both runs.

Table 2: Five models, same prompt, same test cases, two machines

		Score out of 21		**Time per alert**	
Model	**Size**	**CPU**	**GPU**	**CPU**	**GPU**
`mistral:7b`	4.4 GB	**13**	**13**	68.2 s	2.5 s
`phi4-mini`	2.5 GB	**13**	12	24.1 s	3.5 s
`llama3.1:8b`	4.9 GB	12	12	47.5 s	2.1 s
`qwen2.5:7b`	4.7 GB	10	10	43.2 s	2.1 s
`llama3.2:3b`	2.0 GB	5	6	19.6 s	1.4 s

4.1 A bigger model is not a better model

This is the result that surprised us most, and it is the reason this section exists.

Ordered by size, the scores go 5, 13, 13, 10, 12. There is no pattern. The largest model we tested, `llama3.1:8b` with eight billion parameters, did not win. The two seven billion parameter models, `mistral:7b` and `qwen2.5:7b`, are the same size and scored thirty percent apart.

The common advice when running models locally is to pick the biggest one that fits in memory. On our task that advice would have been wrong. What separates these models is not how many parameters they have but what they were trained on and how they were tuned to follow instructions.

We think the reason is that our task is narrow. We are not asking the model to write an essay or reason through a long problem. We ask it to read roughly fifteen lines of log data and fill in

four fields. That kind of task rewards precision and instruction following, and those do not scale with parameter count.

4.2 The smallest model stopped working entirely

`llama3.2:3b` returned *the same answer for all seven alerts.* Every time it said false positive, severity low, technique T1003.002, whatever we gave it.

It is tempting to record this as twenty four percent accuracy. That would be misleading. The model is not partly right, it is not reading the input at all. It scored anything above zero only because one of the seven cases genuinely is T1003.002, so it was accidentally correct once.

This tells us there is a floor below which a model cannot do this task at all, somewhere between three and four billion parameters for this family. We only saw it because we deliberately included a model we expected to be too small. Its very low response time is worthless, since speed means nothing when the answer ignores the question.

4.3 Different models are good at different parts

`qwen2.5:7b` was the worst model at spotting false positives, three out of seven, but the best at identifying the MITRE technique, four out of seven. Capability is not one single number. For us the false positive column matters most, because that is the judgement that can lead to blocking traffic, so we weighted it accordingly when choosing.

5 Experiment 2: the prompt matters more than the model

Having picked `mistral:7b`, we varied how we asked rather than what we asked. Two things changed:

- **Few-shot examples.** Instead of only instructions, we included two complete worked examples in the prompt, one harmless alert and one real attack, each with the correct answer.
- **A restricted answer list.** Instead of letting the model write any MITRE identifier it liked, we gave it a fixed list and made anything outside that list impossible to produce.

Table 3: Same model, four configurations (measured on the laptop CPU)

Examples	Answer list	False pos.	Severity	Technique	Total
few-shot	restricted	7/7	**6/7**	**6/7**	**19/21 (90%)**
few-shot	free	7/7	5/7	5/7	17/21 (81%)
zero-shot	restricted	7/7	4/7	3/7	14/21 (67%)
zero-shot	free	7/7	4/7	2/7	13/21 (62%)

The comparison worth making is this one:

Changing from the second best model to the best model	**+1 point**
Changing from the worst prompt to the best prompt	**+6 points**

Working on the prompt was six times more useful than working on the choice of model. For a project with a deadline this is the single most practical thing we learned. Time spent comparing models has a much lower return than time spent on how the question is asked.

5.1 Why the restricted answer list helps

Without it, the models invented technique identifiers that look real but do not exist, such as T1132.002 and T1048.007. A reader who does not know the ATT&CK catalogue by heart would not notice. Restricting the list makes an invalid answer physically impossible to generate.

It does not make the model understand anything, though. It only removes one class of error. The model still has to choose the right entry, which is why the restriction only pays off properly when combined with the worked examples.

5.2 The model never says it does not know

We deliberately added an OTHER option to the list, plus a field where the model could explain its reasoning, so that an unfamiliar attack could be flagged for a human instead of forced into a wrong category.

OTHER was chosen zero times out of twenty one opportunities. The model always produced a confident answer, including when it was wrong.

This has a practical consequence for anyone building on this. You cannot rely on a model telling you when it is unsure. If you need to detect uncertainty you have to measure it from outside, for example from the confidence value or by comparing two models, and not by politely giving the model a way to opt out.

6 Experiment 3: larger and newer models on better hardware

Experiments 1 and 2 ran on a processor, where a very large model was not practical to test. Once we had a machine with a graphics card and 24 GB of video memory, we could finally answer the obvious objection: was `mistral:7b` chosen because it was the best, or because it was the only thing our hardware could run?

To avoid choosing opponents we expected to beat, we took the candidate list from a public recommendation of the best local models for security work rather than picking them ourselves.

6.1 Two of the recommended models do not fit

Model	Weights	Runs on a 24 GB card?
`qwen3.6:35b` (the top recommendation)	23.9 GB	No
`llama3.3:70b`	42.5 GB	No

The most recommended model needs 23.9 GB of weights on a card with 24 GB, which leaves nothing for the working memory the model needs while running. It would have to push part of itself back onto the processor, which removes the whole benefit of the graphics card.

This is worth stating plainly. General recommendations for the best model are written without any reference to what hardware the reader actually has.

6.2 Results

All models below were tested with the same prompt, the same restricted answer list and the same seven cases.

Table 4: Larger and newer models against the current one

Model	Size	JSON	False pos.	Sev.	Total	Time
`mistral:7b`	**4.4 GB**	7/7	**7/7**	5/7	**19/21**	2.8 s
`qwen3.5:27b`	17 GB	7/7	7/7	3/7	14/21	48.9 s
`qwen3-coder:30b`	18 GB	7/7	5/7	4/7	14/21	2.4 s
`gemma4:12b`	7.6 GB	7/7	6/7	3/7	12/21	57.6 s
`qwen3:30b-a3b`	18 GB	**6/7**	3/7	2/7	10/21	23.5 s

Every one of them lost, and not narrowly. The model we already had is four times smaller than the closest competitors and scored five points higher.

This is the second time the same result appeared. In Experiment 1 the biggest model did not win. Here, with a newer generation of models, more memory available and candidates selected by somebody else, the smallest model won again. We consider this the strongest single result in the report, because it would have been easy for it to go the other way.

6.3 Three failures worth describing

Aggregate scores hide what actually went wrong. Three examples show why we do not simply trust a higher benchmark number.

A model that contradicted itself. On the port scan case, `qwen3.5:27b` filled the technique field with T1547.001, which is about programs starting automatically at boot and has nothing to do with scanning. In the same answer, its own explanation said the activity matched T1592, gathering information about a target. The correct answer was T1046. Three different techniques in one response.

The restricted list guarantees a valid answer. It does not guarantee a considered one. When the model's preferred answer is not on the list, it does not complain, it quietly substitutes something that is on the list, even when that contradicts its own explanation. Anyone reading only the technique field would never see this.

A model that invented the source of an attack. On the same case, `gemma4:12b` wrote that an internal administrator on our VPN had run the scan and marked the whole alert as a false positive. The scan came from outside. The model had taken the description of our network from the prompt and applied it to traffic that did not match it.

This is the more dangerous direction of error. A false alarm wastes an analyst's time. This kind of mistake makes a real attack disappear silently, and no format restriction can catch it.

A model that returned nothing at all. `qwen3:30b-a3b` produced an empty response on one case after waiting almost forty six seconds. It was not malformed output, there was simply no output.

We had previously concluded that constrained decoding was completely reliable, because across fifty six earlier runs every answer was valid. That conclusion was too strong and this result corrects it. The guarantee is that *if* the model produces something, it will match the required shape. There is no guarantee that it produces anything. Any system built on this has to handle an empty answer as a normal possibility.

7 Processor against graphics card

The same model, the same prompt and the same test cases were run on three machines. Only the hardware changed. The model file was byte for byte identical on all three, which we verified by checksum.

Table 5: Same work, three machines

Machine	Processing	Cores or card	Time per alert	Score
Laptop virtual machine	CPU only	4 cores	74.1 s	19/21
AWS `m7i-flex.large`	CPU only	about 1 core	116.1 s	19/21
Google Cloud `g2-standard-4`	**NVIDIA L4**	24 GB card	**2.5 s**	19/21

The graphics card version is about seventy times faster than the AWS processor machine, and the score did not change.

This matters more than it looks. Before this test, one full alert took our pipeline 514 seconds from start to finish, and the backlog of 206 unprocessed alerts in TheHive would have needed about twenty eight hours of computation. On the graphics card the same backlog takes roughly twenty minutes. The delay was a hardware problem, not a problem with our design.

7.1 Two details that are easy to get wrong

The first request is slow. Loading the model into the card takes about thirty seven seconds. After that, answers take two and a half seconds. If the machine sits idle the model is unloaded and the next request pays the loading cost again. For a live demonstration the model has to be warmed up first.

Identical scores do not mean identical answers. The three machines all scored 19 out of 21, but they did not fail on the same cases. The processor version got the severity right on case 01 and the technique wrong on case 03. The graphics card version did the opposite. The model gives exactly the same answer every time on the same machine, but not across machines, because the two kinds of hardware compute in slightly different ways and close decisions can tip either way.

The practical conclusion is that a benchmark result belongs to a combination of model, prompt, answer format *and* hardware. Changing the machine means the benchmark should be run again rather than assumed to still hold.

8 What we chose and why

We use `mistral:7b`.

- It scored highest overall, 19 out of 21.
- It got every false positive right, seven out of seven. This is the judgement we care about most, because it is the one that can lead to blocking traffic.
- It got every MITRE technique right, seven out of seven.
- It answers in about 2.5 seconds on the graphics card.
- At 4.4 GB it leaves most of the card free, so there is room to grow later.

It is not the fastest model available. `phi4-mini` is smaller and was quicker on the processor. We chose accuracy on false positives over speed, because a wrong answer here has a real cost and a slightly slower answer does not.

The important point is that this is now a tested decision rather than a forced one. We gave the model a graphics card with five times the memory it needs and four larger, newer competitors chosen by somebody else, and it still won.

9 What we would not claim

- **Seven cases is a small sample.** A single case is worth almost five percent. The ranking between models is clear, the exact percentages are not precise.
- **Each model saw each case once.** We have no measure of variation. Answers are repeatable on the same machine, but that does not tell us how close the model was to answering differently.
- **The prompt was tuned for `mistral:7b`.** The competitors in Experiment 3 were tested using a prompt developed for the model they were competing against. This probably works against them, and a fairer comparison would tune the prompt separately for each. We did not do that.
- **The processor comparison was never like for like.** The laptop had four real cores and the AWS machine about one. We wanted a machine with four cores for a fair test but it was not available on the account we had.
- **The test set and the prompt share an author.** That is a bias, and the results should be read with it in mind.
- **Two recommended models were excluded on memory, not beaten.** `qwen3.6:35b` and `llama3.3:70b` were never tested.

10 Conclusion

Three results are worth carrying forward.

Size did not predict quality. A 4.4 GB model beat competitors up to 18 GB, twice, on two different generations of model and two different machines. For a narrow task with a fixed answer format, how a model was trained matters more than how large it is.

How you ask beats what you ask. Improving the prompt was worth six points. Choosing a better model was worth one. If we had to spend our remaining time on one of the two, it would be the prompt.

Hardware changed the speed and not the answer. Moving to a graphics card made the system about seventy times faster with the same score, which turned a pipeline that was too slow to use into one that responds in a few seconds.

The most useful thing we found, though, was not in the scores. It was watching a model recommend blocking one of our own machines, and watching another one describe an external attacker as an internal administrator. Neither mistake shows up as invalid output, and neither would be caught by a stricter answer format. That is the reason the decision to block anything sits in ordinary code that we can read and test, and never in the model.